Kafka Distributed Messaging Engine for Real Time Chat Application

A PROJECT REPORT

submitted by

ATUL KUMAR (22190503014)

to

the Central University of Jharkhand, Ranchi

in fulfillment of the Engineering Project

of

Integrated B. Tech and M. Tech
in
Computer Science and Engineering

Department of Computer Science and Engineering

CENTRAL UNIVERSITY OF JHARKHAND

Cheri-Manatu, Ranchi- 835222

DECLARATION

I’m saying this project - 'Kafka Distributed Messaging Engine for Real Time Chat Application' - is something I made on my own. It counts toward Engineering Project for my B. Tech in CSE at Central University of Jharkhand. The work happened while being guided by Prof. S. C. Yadav, Head of CSE Department. This write-up shows what I did personally.

I want to say straight up - this piece wasn’t turned in earlier for a credential, diploma, or academic credit at any other institution.

Date: 17/04/2026

Place: Ranchi.

Student Name: Atul Kumar

Roll Number: 22190503014

Signature: Atul Kumar

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CENTRAL UNIVERSITY OF JHARKHAND

CHERI-MANATU, RANCHI

CERTIFICATE

This paper shows the project called “Kafka Distributed Messaging Engine for Real Time Chat Application” was finished by Atul Kumar, Roll No. 22190503014, from the Computer Science and Engineering department at Central University of Jharkhand; it’s original work carried out during 2025–2026 under my supervision.

This work fulfills one piece required for a Bachelor of Tech in Computer Science and Engineering. During development, attention went to hands-on activities linked with the class. Even though it is still unfinished, it helps reach graduation goals. As part of school demands, specific rules needed to be met. Fair enough, care was taken to match each rule from the team. Still, there’s space to tweak things later once they’re handed in.

Project Guide

Name: Prof. S. C. Yadav

Signature:

Head of Department

Name: Prof. S. C. Yadav

Signature:

ACKNOWLEDGEMENT

I'm thankful to my project guide, Prof. S. C. Yadav, Head of CSE Department, without whose help this project - "Kafka Distributed Messaging Engine for Real Time Chat Application" - wouldn't have taken shape. Their advice kept things on track; their feedback sharpened every step. With steady encouragement plus clear direction, they made tough parts easier to handle. This whole effort leaned heavily on their experience and timely input.

I'd like to thank the department head of Computer Science and Engineering at Central University of Jharkhand - thanks to their support and helpful set-up, I could finish this project without major hurdles.

I'd like to thank every faculty member along with lab staff from the CSE department - each one who helped when things were getting set up or tested.

Fair enough, I owe a lot to my folks and friends - their push kept me going, so this thing got done.

ATUL KUMAR

Roll Number: 22190503014

Department of Computer Science and Engineering

Central University of Jharkhand

Ranchi

TABLE OF CONTENTS

Abstract

The rapid growth of real-time communication platforms has created a strong demand for backend systems that are highly scalable, fault-tolerant, and capable of handling large volumes of concurrent data streams. Traditional chat systems based on synchronous request-response models and centralized databases often face limitations in terms of scalability, latency, and system reliability. To address these challenges, this project presents the design and implementation of a custom distributed event streaming system for real-time chat applications, inspired by the architectural principles of Apache Kafka.

The proposed system is developed entirely in Java (JDK 11) and features a lightweight REST API Gateway implemented using the Javalin framework, which acts as the entry point for handling client requests. At the core of the system lies a custom-built distributed messaging engine that supports topic-based message streaming, partitioning, and replication across multiple nodes. Unlike conventional systems that rely on relational or NoSQL databases, this project implements a high-performance storage mechanism based on append-only log files using Java NIO File-Channel, ensuring efficient disk I/O operations and data durability.

To enable distributed coordination and fault tolerance, the system integrates Apache Zookeeper, which manages partition leadership, broker coordination, and consumer group assignments. Additionally, a tiered storage mechanism is incorporated to optimize storage efficiency by archiving older log segments into a simulated cloud environment. On the frontend, a responsive chat interface is developed using HTML, Vanilla JavaScript, and Tailwind CSS, providing users with a modern messaging experience through asynchronous polling.

The system demonstrates high throughput, low latency, and reliable message delivery under continuous operation. Experimental results confirm that the architecture effectively supports real-time communication while maintaining data consistency and system stability. This project highlights the feasibility and effectiveness of building a distributed messaging platform from scratch and provides a strong foundation for developing large-scale, real-world streaming applications.

Chapter 01

INTRODUCTION

1.1 Background

In recent years, the demand for real-time communication systems has increased significantly due to the widespread adoption of social media platforms, instant messaging applications, and distributed web services. These applications require backend systems capable of handling high volumes of concurrent user interactions while maintaining low latency, high availability, and reliable data delivery. Traditional monolithic architectures and synchronous request-response communication models are often inadequate for such scenarios, as they introduce bottlenecks in performance and limit system scalability.

To overcome these challenges, modern systems have shifted toward distributed and event-driven architectures, where components operate independently and communicate asynchronously through message streams. Distributed messaging systems play a crucial role in this paradigm by enabling efficient data exchange between producers and consumers without tight coupling. Technologies such as Apache Kafka have popularized the concept of log-based messaging systems, where data is stored as a sequence of immutable records and processed in real time by multiple consumers.

Inspired by these principles, this project focuses on the development of a custom distributed event streaming system designed specifically for real-time chat applications. Instead of relying on existing messaging platforms, the system is built entirely from scratch using Java, providing a deeper understanding of core distributed system concepts such as partitioning, replication, fault tolerance, and asynchronous data processing. The architecture replaces traditional database storage with an append-only log mechanism, enabling high-throughput disk operations and efficient message persistence.

Furthermore, the integration of distributed coordination tools allows the system to manage multiple nodes, assign responsibilities dynamically, and maintain consistency across the cluster. By combining these elements, the project demonstrates how scalable and reliable backend systems for real-time communication can be engineered using fundamental distributed system principles. This approach not only enhances system performance but also provides flexibility for future expansion into large-scale, production-level deployments.

1.2 Problem Definition

Despite the rapid growth of real-time communication platforms, many existing chat backend systems still rely on traditional architectures that are not optimized for high scalability and continuous data streaming. Most conventional systems are built using synchronous request-response models, where each user interaction directly depends on server availability and response time. This approach introduces latency, limits throughput, and creates performance bottlenecks under high user load.

Another major limitation lies in the use of centralized database systems for storing chat messages. Relational and even some NoSQL databases are not inherently designed for high-frequency, append-heavy workloads typical of real-time messaging systems. As the volume of messages increases, these systems often suffer from increased latency, complex scaling requirements, and reduced efficiency in handling concurrent read-write operations.

Additionally, existing solutions often exhibit tight coupling between system components such as message producers, storage layers, and consumers. This lack of decoupling reduces system flexibility and makes it difficult to scale individual components independently. Fault tolerance is also a concern, as failures in one part of the system can disrupt the entire communication pipeline.

From a learning and engineering perspective, another gap exists in understanding how modern distributed messaging platforms function internally. Most implementations rely on pre-built systems like Apache Kafka without exploring the underlying mechanisms such as partitioned logs, replication strategies, offset management, and distributed coordination.

Therefore, the problem addressed in this project is the design and development of a scalable, fault-tolerant, and high-performance real-time messaging backend that:

- Eliminates dependency on traditional database systems
- Supports asynchronous communication through event streaming
- Implements distributed system concepts such as partitioning and replication
- Provides efficient and durable message storage using append-only logs
- Enables independent scaling of system components
- Demonstrates a ground-up implementation of a Kafka-like messaging system

By addressing these challenges, the project aims to build a robust backend architecture capable of supporting modern real-time communication requirements while also providing a deep understanding of distributed system design principles.

1.3 Aims and Objectives

The primary aim of this project is to design and implement a scalable, distributed, and fault-tolerant real-time chat backend system using a custom-built event streaming engine inspired by modern messaging platforms such as Apache Kafka. The system focuses on achieving high throughput, low latency, and reliable message delivery by leveraging asynchronous communication and log-based storage mechanisms.

Unlike conventional systems that depend on pre-built frameworks and databases, this project emphasizes building core distributed system components from scratch to gain a deeper understanding of internal architectural processes and system behaviour under real-world conditions.

Objectives

- To design and develop a custom distributed messaging engine using Java that supports topic-based communication and asynchronous message streaming.
- To implement partitioned, append-only log storage using Java NIO File-Channel for efficient and durable message persistence.
- To develop a lightweight REST API Gateway using the Javalin framework for handling client requests and routing messages to the backend system.
- To integrate Apache Zookeeper for distributed coordination, including leader election, partition management, and consumer group assignment.
- To design a system architecture that ensures loose coupling between producers, brokers, and consumers, enabling independent scalability of each component.
- To implement a tiered storage mechanism that archives older log segments into a simulated cloud environment for optimized storage management.

1.4 Scope of the Project

The scope of this project focuses on the complete design and implementation of a custom distributed event streaming system tailored for real-time chat applications. The system is developed as a fully functional prototype that demonstrates how scalable messaging backends can be built from scratch using core distributed system principles, without relying on existing frameworks such as Apache Kafka.

The project encompasses the development of a multi-layered architecture that includes a REST-based API Gateway, a custom-built messaging engine, a log-based storage system, distributed coordination using Apache Zookeeper, and a responsive frontend chat interface. Each component is designed to operate independently while maintaining seamless interaction through asynchronous communication.

At the core of the system is a custom messaging engine that supports topic-based message streaming, partitioned data storage, and consumer-based message retrieval. Messages are stored using an append-only log mechanism implemented through Java NIO File-Channel, ensuring efficient disk I/O operations and high-throughput data handling. This approach eliminates the dependency on traditional relational or NoSQL databases, making the system more suitable for streaming workloads.

The system also integrates Apache Zookeeper to manage distributed coordination tasks such as broker registration, partition leadership, and consumer group management. This enables the system to simulate real-world distributed environments and ensures fault tolerance and consistency across multiple nodes.

On the user interaction side, a modern chat interface is developed using HTML, JavaScript, and Tailwind CSS, which communicates with the backend through asynchronous polling. This allows users to send and receive messages in near real-time, demonstrating the practical application of the messaging system.

Additionally, the project includes a tiered storage mechanism that archives older log segments into a simulated cloud storage environment, improving storage efficiency and reflecting real-world data lifecycle management strategies.

The scope of this project is limited to the implementation of a prototype-level distributed messaging system suitable for real-time chat applications. While it demonstrates key concepts such as partitioning, replication, asynchronous communication, and log-based storage, it does not include production-level deployment features such as full-scale cloud orchestration, advanced security mechanisms, or large-scale cluster management.

Overall, the project provides a comprehensive understanding of how modern distributed messaging systems operate and serves as a strong foundation for further enhancements toward production-grade real-time communication platforms.

Chapter 02

SYSTEM REQUIREMENTS AND METHODOLOGY

2.1 Requirement Analysis

Requirement analysis is a critical phase in system development that defines what the system must accomplish and the constraints under which it operates. For this project, the requirements are derived based on the need to build a scalable, distributed, and real-time messaging backend capable of handling chat communication efficiently without relying on traditional database systems.

The system is designed around an event-driven architecture where messages are produced, stored, and consumed asynchronously. The frontend interacts with the backend through REST APIs, while the backend processes and persists data using a custom messaging engine and log-based storage mechanism. The analysis includes functional requirements, non-functional requirements, and system-level requirements covering both software components and runtime behaviour.

A. Functional Requirements

- The system should allow users to send and receive chat messages in real time through a web-based interface.
- The API Gateway should accept HTTP requests from the frontend and route them to the messaging engine for processing.
- The messaging engine should support topic-based communication, enabling logical separation of message streams.
- The system should implement a producer-consumer model where messages are published by producers and consumed by multiple consumers.
- Messages should be stored using an append-only log mechanism to ensure durability and sequential consistency.
- The system should support partitioned message storage to distribute load and improve scalability.
- Consumers should be able to read messages based on offsets, ensuring ordered message retrieval.
- The system should maintain metadata related to topics, partitions, and consumer groups.
- The system should provide APIs to fetch stored messages for frontend visualization.
- The system should archive older log segments using a tiered storage mechanism for efficient storage management.

B. Non-Functional Requirements

- The system should achieve high throughput by efficiently handling a large number of messages per second.
- The system should maintain low latency in message delivery to support near real-time communication.

- The system should ensure fault tolerance through replication and distributed coordination mechanisms.
- The system should be scalable, allowing additional nodes or partitions to be added without affecting performance.
- The storage mechanism should be optimized for sequential disk writes to improve I/O efficiency.
- The system should maintain data durability, ensuring that messages are not lost even in case of failures.
- The system should be modular, allowing independent development and scaling of components such as API Gateway, messaging engine, and storage.
- The frontend interface should be responsive and capable of handling continuous updates without performance degradation.

C. Software Requirements

The system is developed using a combination of backend, coordination, and frontend technologies.

- Java (JDK 11) for implementing the core messaging engine and backend logic.
- Javalin framework for building the lightweight REST API Gateway.
- Apache Zookeeper (v3.8.4) for distributed coordination and cluster management.
- Java NIO File-Channel for implementing high-performance append-only log storage.
- Vite development server for running the frontend application.
- HTML, Vanilla JavaScript, and Tailwind CSS for building the chat user interface.

D. System Level Requirements

These requirements define how different components interact within the system.

- The frontend sends chat messages to the API Gateway through HTTP requests.
- The API Gateway processes the request and forwards the message to the messaging engine.
- The messaging engine writes the message to a partitioned append-only log using File-Channel.
- Zookeeper manages metadata such as partition ownership and consumer group coordination.
- Consumers read messages from log files based on offsets and deliver them to the frontend.
- The frontend continuously polls the backend to fetch new messages and update the user interface.
- The tiered storage module periodically moves older log segments to a simulated cloud storage directory.

2.2 System Design

The architecture of the system is divided into multiple layers, each responsible for a specific function in the message processing pipeline. The design follows a producer-broker-consumer model adapted for a chat application.

A. System Architecture→

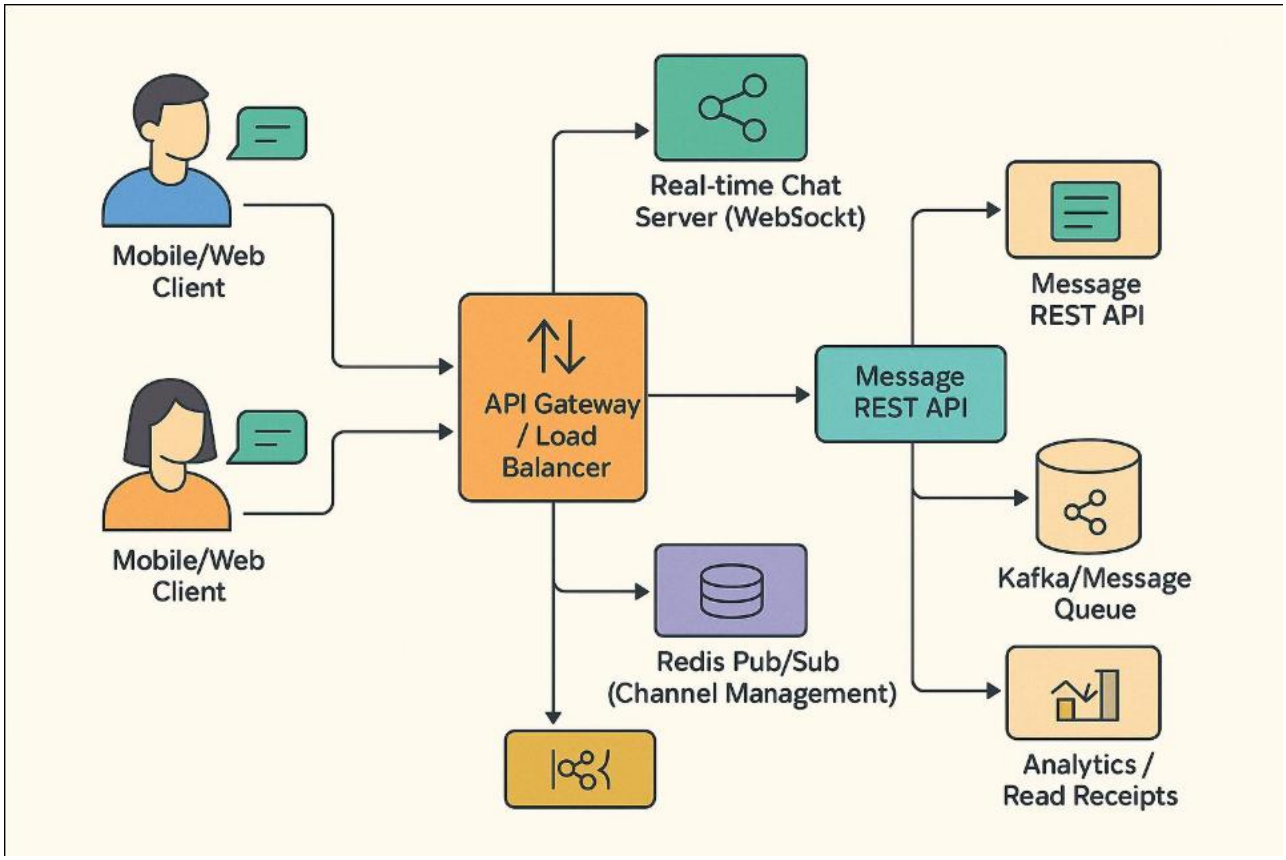


Figure 2.1: System Architecture of the proposed Distributed Messaging System.

The above diagram illustrates the overall architecture of the proposed distributed messaging system. The frontend acts as the message producer, sending requests to the API Gateway. The API Gateway forwards these messages to the custom messaging engine, which processes and assigns them to specific topics and partitions. Messages are stored in append-only log files using File-Channel, while Zookeeper manages coordination tasks such as leader election and consumer group assignment. Consumers retrieve messages based on offsets, and the frontend continuously polls the backend to display real-time updates.

Component Description:

- **Frontend Layer:**
  The user interface is built using HTML, JavaScript, and Tailwind CSS. It allows users to send and receive messages. The frontend communicates with the backend through HTTP requests and continuously polls for new messages.
- **API Gateway (Javalin):**

Acts as the entry point for all client requests. It handles HTTP traffic, validates input data, and forwards messages to the messaging engine. It also provides endpoints for retrieving stored messages.

- **Messaging Engine (Custom Broker):**
  This is the core component of the system. It manages topics, partitions, and message routing. It ensures that messages are correctly written to log storage and made available to consumers.
- **Log Storage (Append-Only Logs):**
  Messages are stored in partitioned log files using Java NIO File-Channel. Each message is appended sequentially, ensuring high write throughput and durability.
- **Distributed Coordination (Zookeeper):**
  Zookeeper manages cluster coordination tasks such as broker registration, partition leadership, and consumer group assignment. It ensures consistency and fault tolerance in a distributed environment.
- **Consumer Layer:**
  Consumers read messages from log files using offset-based access. They process messages and make them available for frontend retrieval.

B. Data Flow Diagram

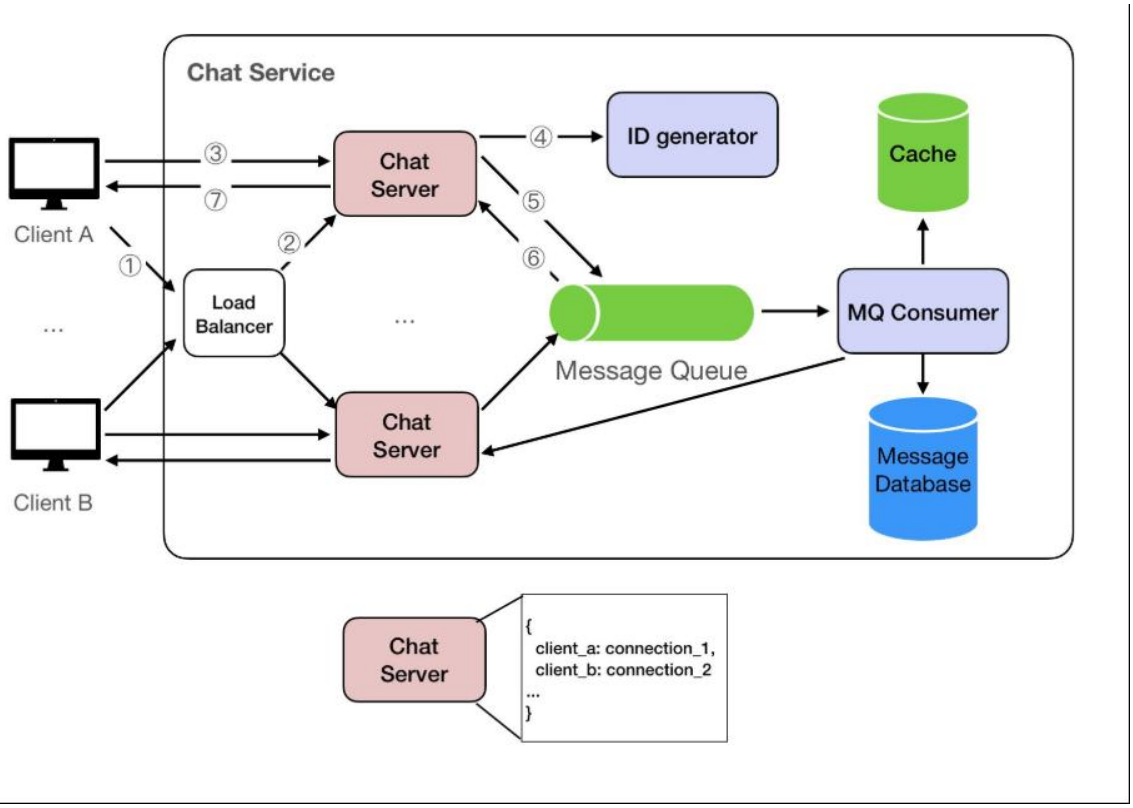


Figure 2.2: Data Flow Diagram of the Messaging System.

The data flow diagram represents the movement of messages across different components of the system. Messages are generated at the frontend and transmitted to the API Gateway, which forwards them to the messaging engine. The engine assigns

messages to partitions and stores them in log files. Zookeeper updates system metadata, while consumers retrieve messages and deliver them back to the frontend. This flow ensures asynchronous and efficient communication.

The system follows a well-defined data flow from message generation to delivery.

Process Flow:

1. The user sends a message through the frontend interface.
2. The message is sent as an HTTP request to the API Gateway.
3. The API Gateway forwards the request to the messaging engine.
4. The messaging engine assigns the message to a specific topic and partition.
5. The message is written to the append-only log file using File-Channel.
6. Zookeeper updates metadata related to partitions and consumers.
7. Consumers read new messages based on offsets.
8. The frontend polls the API and retrieves the latest messages.
9. The chat interface updates dynamically to reflect new messages.

This asynchronous flow ensures that message production and consumption are decoupled, improving system performance and scalability.

C. STORAGE DESIGN:

The storage system is designed to handle high-throughput data ingestion efficiently.

- Messages are stored in **append-only log segments**.
- Each partition maintains its own .log file for message data.
- .index files are used to map message offsets for quick retrieval.
- Sequential writes reduce disk seek time and improve performance.
- A **tiered storage mechanism** moves older log segments to a simulated cloud storage directory (.cloud_bucket/).

This design eliminates the need for traditional databases and aligns with modern event streaming architectures.

D. COORDINATION AND PARTITIONING DESIGN:

To support distributed operation, the system implements coordination and partition management.

• Topics are divided into multiple partitions to distribute load.
• Each partition has a leader responsible for handling read/write operations.
• Zookeeper maintains metadata about:

- Active brokers

- Partition leaders
- Consumer group assignments

• Consumers are assigned partitions dynamically, ensuring balanced workload distribution.

This design enables horizontal scalability and improves system reliability.

E. SDLC METHOD MAPPING:

The system design follows a structured Software Development Life Cycle (SDLC):

- Requirement Analysis → Identification of system needs.
- Design Phase → Architecture and component modelling.
- Implementation Phase → Development of modules (engine, API, storage, UI).
- Testing Phase → Validation of system functionality and performance.

Each phase contributes to building a robust and scalable distributed messaging system.

F. BLOCK REPRESENTATION OF THE SYSTEM:

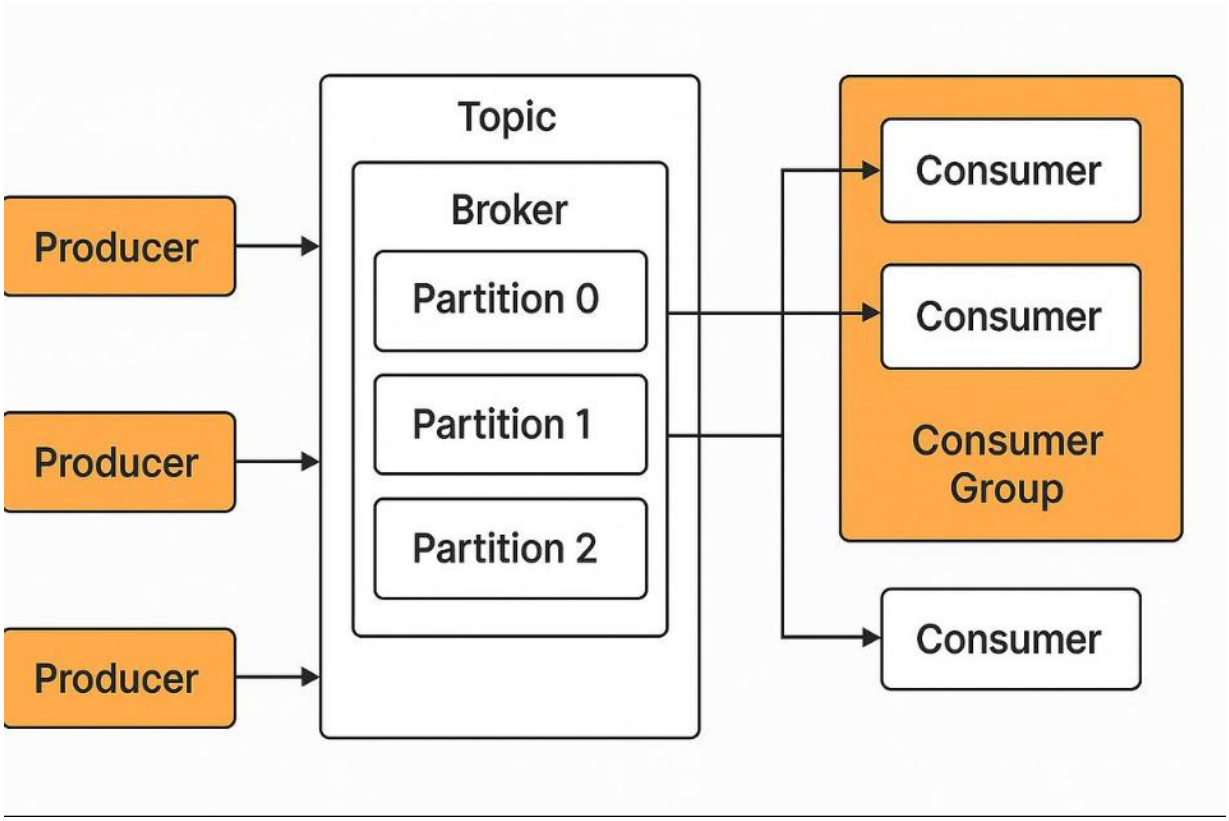


Figure 2.3: Block Representation of the Distributed System.

The block diagram presents a high-level modular view of the system. The input layer consists of the frontend interface, followed by the processing layer comprising the API Gateway and messaging engine. The storage layer handles message persistence using append-only logs, while the coordination layer manages distributed operations using Zookeeper. The output layer delivers messages back to the frontend through consumers.

Chapter 03

IMPLEMENTATION PHASE

The implementation phase focuses on the practical realization of the proposed distributed messaging system for real-time chat applications. In this phase, the conceptual design and architectural decisions discussed earlier are translated into working system components using appropriate technologies and programming techniques. The implementation is carried out in a modular manner, where each component is developed independently while ensuring seamless integration with other parts of the system.

3.1 Messaging Engine Module (Custom Kafka-like Broker)

The Messaging Engine Module forms the core component of the proposed system and acts as a custom-built distributed event streaming broker. This module is responsible for managing message flow between producers and consumers, handling topic-based communication, organizing data into partitions, and ensuring reliable message delivery. Unlike traditional implementations that rely on pre-built platforms such as Apache Kafka, this project implements the entire messaging engine from scratch using Java, providing full control over system behaviour and internal data handling.

3.1.1 Core Functionality

The messaging engine is designed based on the **producer-broker-consumer model**, where it acts as the central broker responsible for receiving, storing, and distributing messages.

- It accepts messages from producers via the API Gateway.
- It categorizes messages into logical channels called **topics**.
- Each topic is further divided into **partitions** to enable parallel processing and scalability.
- It ensures that messages are written sequentially to storage for durability.
- It allows consumers to read messages independently based on offsets.

This design ensures that producers and consumers remain decoupled, improving system flexibility and scalability.

3.1.2 Topic and Partition Management

The system organizes messages using a hierarchical structure:

- **Topics:**
  Topics act as logical streams of data. Each chat channel or conversation can be mapped to a topic, allowing structured message organization.

- **Partitions:**
  Each topic is divided into multiple partitions. Messages are distributed across these partitions to balance load and enable parallel consumption.
- **Partition Assignment:**
  When a message is received, the messaging engine determines the target partition using a hashing or round-robin mechanism. This ensures even distribution of data.

This partitioned design is critical for achieving high throughput and horizontal scalability.

3.1.3 Message Handling and Routing

When a message enters the system:

1. The API Gateway forwards the message to the messaging engine.
2. The messaging engine validates and processes the message.
3. The message is assigned to a specific topic and partition.
4. It is then forwarded to the storage module for persistence.
5. Metadata such as offset and timestamp are updated.

This pipeline ensures that messages are handled efficiently while maintaining order within each partition.

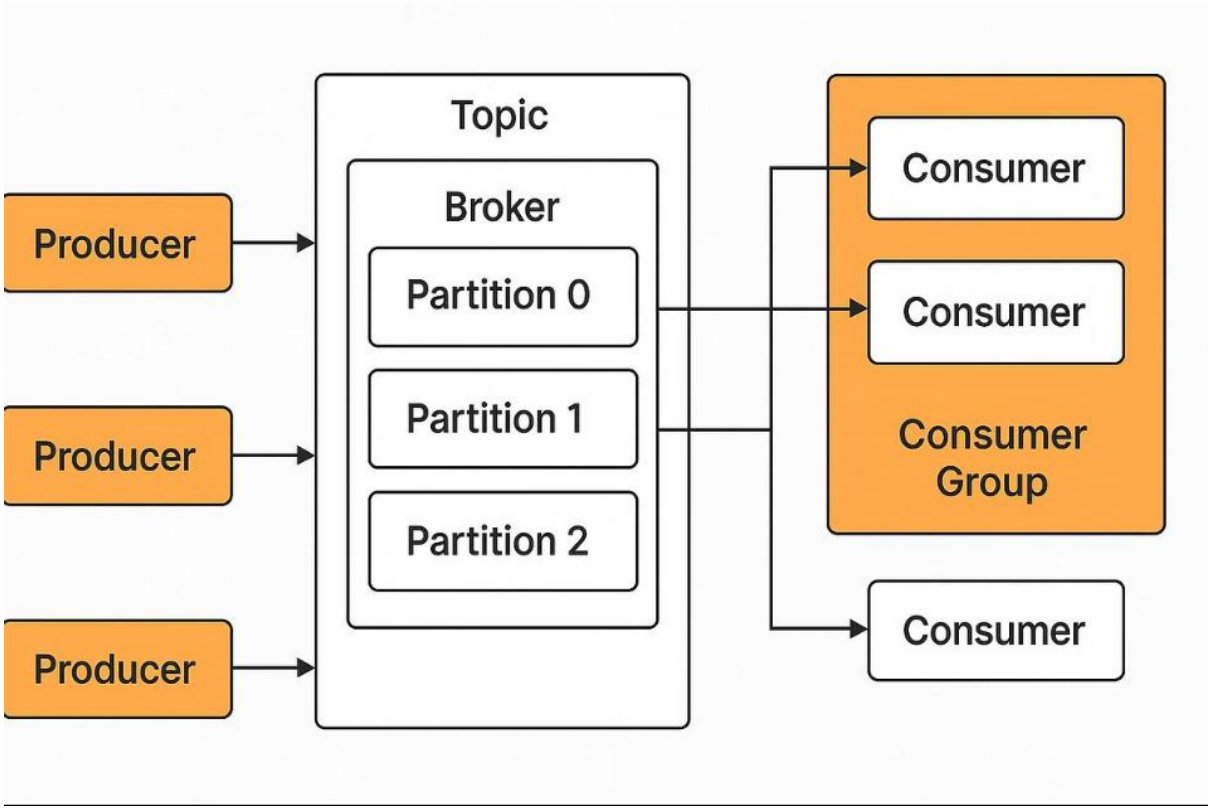


Figure 3.1: Message Processing Flow in the Messaging Engine.

The flowchart illustrates how messages are processed within the messaging engine. Messages received from the API Gateway are validated and assigned to specific topics. Based on partitioning logic, each message is directed to a corresponding partition, where

it is stored in append-only logs. The system then updates offsets and makes the message available for consumer retrieval.

3.1.4 Offset Management

The messaging engine maintains an offset-based indexing system, where each message within a partition is assigned a unique sequential identifier.

- Offsets allow consumers to track their reading position.
- Consumers can resume reading from a specific offset in case of failure.
- This ensures fault tolerance and message replay capability.

Offset tracking is a key feature that differentiates log-based systems from traditional queue-based messaging systems.

3.1.5 Replication and Fault Tolerance

To improve reliability, the messaging engine supports **partition replication** across multiple nodes.

- Each partition has a **leader node** responsible for handling read/write operations.
- Replica nodes maintain copies of the data for backup.
- In case of leader failure, a new leader is elected using Zookeeper coordination.

This mechanism ensures that the system remains operational even during node failures.

3.1.6 Concurrency and Performance Handling

The messaging engine is designed to handle concurrent message processing efficiently.

- Multi-threading is used to handle multiple producer and consumer requests simultaneously.
- Partition-level parallelism allows independent processing of message streams.
- Sequential disk writes reduce overhead and improve performance.

This results in high throughput and low latency, which are essential for real-time chat systems.

3.1.7 Integration with Other Modules

The messaging engine interacts with multiple system components:

- **API Gateway:** Receives incoming messages
- **Log Storage Module:** Persists messages to disk
- **Zookeeper:** Handles coordination and metadata
- **Consumers:** Reads messages for delivery

This integration ensures a seamless flow of data across the system.

Annotation:

This module acts as the **central processing and routing layer** of the system. It is responsible for implementing core distributed messaging functionalities such as topic management, partitioning, offset tracking, and fault tolerance, making it the most critical component of the overall architecture.

3.2 API Gateway Module (Javalin-based REST Server)

The API Gateway Module serves as the entry point for all client interactions within the system. It is responsible for handling incoming HTTP requests from the frontend, processing user inputs, and forwarding them to the underlying messaging engine. This module plays a crucial role in decoupling the client-side interface from the backend processing logic, ensuring a clean separation of concerns and improving system modularity.

The API Gateway is implemented using the Javalin framework, a lightweight and high-performance web framework for Java. It runs on port 8082 and provides RESTful endpoints for message publishing and retrieval. The choice of Javalin allows for minimal overhead, fast request handling, and easy integration with the custom messaging engine.

3.2.1 Core Responsibilities

The API Gateway performs several essential functions within the system:

- Accepts HTTP requests from the frontend chat interface.
- Validates incoming data such as message content, user identification, and topic information.
- Routes messages to the appropriate components of the messaging engine.
- Provides endpoints for fetching stored messages from the backend.
- Handles response formatting and sends structured data back to the client.

This module acts as a bridge between the user interface and the distributed messaging backend.

3.2.2 API Endpoints

The system exposes multiple REST endpoints to facilitate communication between the frontend and backend.

1. Message Publishing Endpoint:

“POST /api/messages”

- Receives chat messages from the frontend in JSON format.
- Extracts fields such as message content, topic, and timestamp.
- Forwards the message to the messaging engine for processing and storage.
- Returns a success response after successful message ingestion.

2. Message Retrieval Endpoint

“GET /api/messages”

- Fetches recent messages from the messaging engine or storage layer.
- Supports optional parameters such as topic filtering and message limits.
- Returns messages in JSON format for frontend rendering.

These endpoints enable asynchronous communication between the frontend and backend components.

3.2.3 Request Processing Flow

The API Gateway follows a structured flow for handling requests:

1. The frontend sends an HTTP request (POST/GET).
2. The API Gateway receives and parses the request.
3. Input validation is performed to ensure data integrity.
4. The request is forwarded to the messaging engine.
5. The messaging engine processes the message and interacts with storage.
6. A response is generated and returned to the frontend.

This workflow ensures efficient and reliable communication across the system.

3.2.4 Input Validation and Error Handling

To maintain system robustness, the API Gateway incorporates validation and error-handling mechanisms:

- Checks for missing or invalid fields in incoming requests.
- Ensures proper JSON structure before processing.
- Handles exceptions such as invalid endpoints or server errors.
- Returns appropriate HTTP status codes (e.g., 200 OK, 400 Bad Request, 500 Internal Server Error).

This prevents malformed data from entering the system and improves reliability.

3.2.5 Performance Considerations

The API Gateway is designed for high performance and scalability:

- Lightweight framework ensures minimal latency.
- Non-blocking request handling supports concurrent users.
- Efficient routing reduces processing overhead.
- Works seamlessly with asynchronous backend processing.

These features allow the system to handle multiple user requests efficiently.

3.2.6 Integration with Other Modules

The API Gateway interacts closely with other system components:

- **Front-end Module:** Receives requests and sends responses.
- **Messaging Engine:** Forwards messages for processing and routing.
- **Storage Module:** Retrieves stored messages indirectly through the engine.

This integration ensures smooth data flow and consistent system behaviour.

Annotation:

This module acts as the **communication interface layer** of the system. It ensures secure, efficient, and structured interaction between the frontend and the distributed messaging backend, enabling real-time message exchange.

3.3 Log Storage Module (File-Channel & Append-Only Logs)

The Log Storage Module is a fundamental component of the proposed system and serves as the primary mechanism for persistent data storage. Unlike traditional systems that rely on relational or NoSQL databases, this project adopts a log-based storage architecture inspired by distributed streaming platforms. The module is designed to handle high-throughput message ingestion efficiently by utilizing append-only log files and low-level file operations through Java NIO File-Channel.

This approach ensures that all messages are stored sequentially on disk, enabling fast write operations, improved data durability, and efficient retrieval based on offsets.

3.3.1 Storage Architecture

The storage system is organized into partitioned log segments, where each partition maintains its own set of files.

1. .log Files:

These files store the actual message data in a sequential, append-only format. Each new message is appended to the end of the file without modifying existing data.

2. .index Files:

These files maintain mappings between message offsets and their physical positions within the log file. This allows fast lookup and retrieval of messages without scanning the entire file.

3. Segment-Based Storage:

Log files are divided into segments of fixed size. Once a segment reaches its size limit, a new segment is created. This helps in efficient memory management and simplifies archival operations.

This structured design ensures that the system can handle continuous data growth without performance degradation.

3.3.2 Append-Only Log Mechanism

The append-only approach is central to the system's storage design.

- Messages are never updated or deleted once written.
- New data is always appended at the end of the log file.
- Sequential writes minimize disk seek time and improve I/O performance.
- The immutability of data ensures consistency and simplifies concurrency handling.

This model is particularly effective for streaming systems where data is continuously generated and consumed.

3.3.3 File-Channel Based Implementation

The storage module leverages Java NIO File-Channel for efficient disk operations.

- File-Channel allows direct interaction with the file system at a lower level.
- Supports high-performance, non-blocking I/O operations.
- Enables writing large volumes of data with minimal overhead.
- Uses Byte-Buffer for efficient memory management during read/write operations.

By using File-Channel instead of traditional file I/O methods, the system achieves significantly better performance and scalability.

3.3.4 Message Persistence Workflow

The process of storing a message in the log system follows these steps:

1. The messaging engine forwards the message to the storage module.
2. The message is serialized into a binary or structured format.
3. The system identifies the target partition and corresponding log segment.
4. The message is appended to the end of the .log file using File-Channel.
5. The offset and position are recorded in the .index file.
6. Metadata such as timestamp and partition ID are updated.

This workflow ensures that all messages are stored reliably and can be retrieved efficiently.

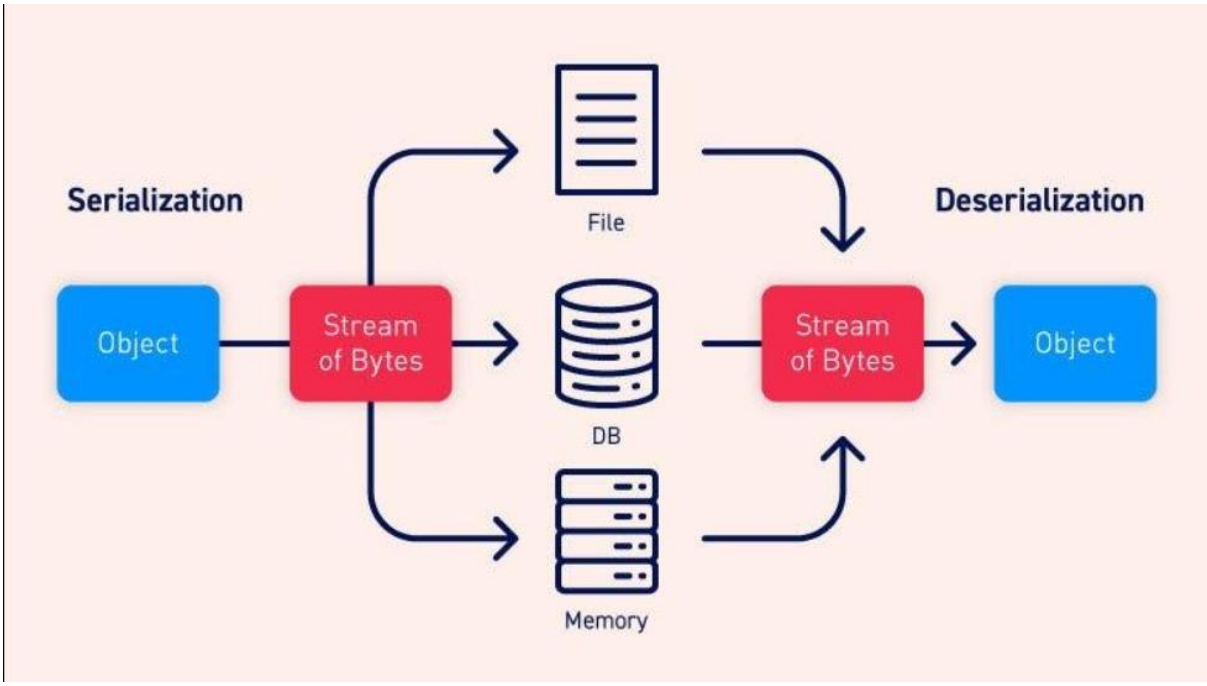


Figure 3.2: Log Storage and Messaging Persistence Workflow.

The flowchart represents the process of storing messages in the append-only log system. Messages are serialized and written to log files using FileChannel. Corresponding index entries are created to map offsets to physical positions, enabling efficient retrieval. This workflow ensures high-performance and reliable data persistence.

3.3.5 Offset-Based Retrieval

The storage system supports efficient message retrieval using offsets.

- Each message is assigned a unique sequential offset within a partition.
- Consumers use offsets to read messages in order.
- The .index file enables direct access to message locations.
- This avoids full file scans and reduces read latency.

Offset-based retrieval is essential for enabling replay, fault recovery, and consistent message consumption.

3.3.6 Performance and Reliability Advantages

The log-based storage design provides several advantages:

- High write throughput due to sequential disk access
- Reduced I/O overhead compared to random writes in databases
- Improved durability as data is written directly to disk
- Simplified concurrency handling due to immutable data
- Efficient scaling with increasing data volume

These features make the storage module highly suitable for real-time messaging systems.

3.3.7 Integration with Other Modules

The Log Storage Module works closely with multiple system components:

- **Messaging Engine:** Sends messages for persistence
- **Consumers:** Reads messages based on offsets
- **Tiered Storage Module:** Archives older log segments
- **Zookeeper (indirectly):** Maintains metadata related to partitions

This integration ensures seamless data flow across the system.

Annotation

This module acts as the **persistent data layer** of the system. It replaces traditional databases with a high-performance, append-only log structure, enabling efficient storage, retrieval, and scalability in a distributed messaging environment.

3.4 Distributed Coordination Module (Zookeeper Integration)

The Distributed Coordination Module is responsible for managing the coordination and synchronization of multiple components in the distributed messaging system. In a multi-node environment, maintaining consistency, fault tolerance, and proper resource allocation requires a centralized coordination mechanism. For this purpose, the system integrates Apache Zookeeper, which acts as a reliable coordination service for distributed applications.

Zookeeper provides a hierarchical namespace and supports distributed synchronization primitives that allow the system to manage metadata, track active nodes, and ensure consistent system behaviour across multiple brokers and consumers.

3.4.1 Role of Zookeeper in the System

Zookeeper acts as the coordination backbone of the system and is responsible for:

- Maintaining metadata about brokers, topics, and partitions
- Managing partition leadership across nodes
- Coordinating consumer group assignments
- Monitoring node health and availability
- Facilitating failover and recovery mechanisms

By centralizing coordination logic, Zookeeper simplifies the management of distributed components and ensures system reliability.

3.4.2 Broker Registration and Discovery

When a messaging broker (node) starts:

- It registers itself with Zookeeper by creating a znode (Zookeeper node).
- This znode contains metadata such as broker ID, network address, and status.
- Other components in the system can discover active brokers by querying Zookeeper.

This dynamic registration mechanism allows the system to adapt to changes in the cluster, such as node addition or removal.

3.4.3 Partition Leadership Management

Each partition in the system is assigned a **leader node** responsible for handling read and write operations.

- Zookeeper maintains information about partition leaders.
- When a broker becomes unavailable, Zookeeper triggers a leader re-election process.
- A replica node is promoted as the new leader to maintain system availability.

This ensures that the system continues to function even in the presence of failures.

3.4.4 Consumer Group Coordination

The system supports multiple consumers that can read messages concurrently.

- Consumers are organized into **consumer groups**.
- Zookeeper assigns partitions to consumers dynamically.
- Each partition is consumed by only one consumer within a group to avoid duplication.
- Load balancing is achieved by distributing partitions evenly among consumers.

This mechanism enables efficient parallel processing and scalability.

3.4.5 Failure Detection and Recovery

Zookeeper continuously monitors the health of nodes using heartbeat mechanisms.

- If a broker fails or disconnects, its corresponding znode is removed.
- Zookeeper detects the failure and triggers recovery actions such as:
  Reassigning partitions
  Electing new leaders
  Updating metadata

This ensures fault tolerance and minimizes system downtime.

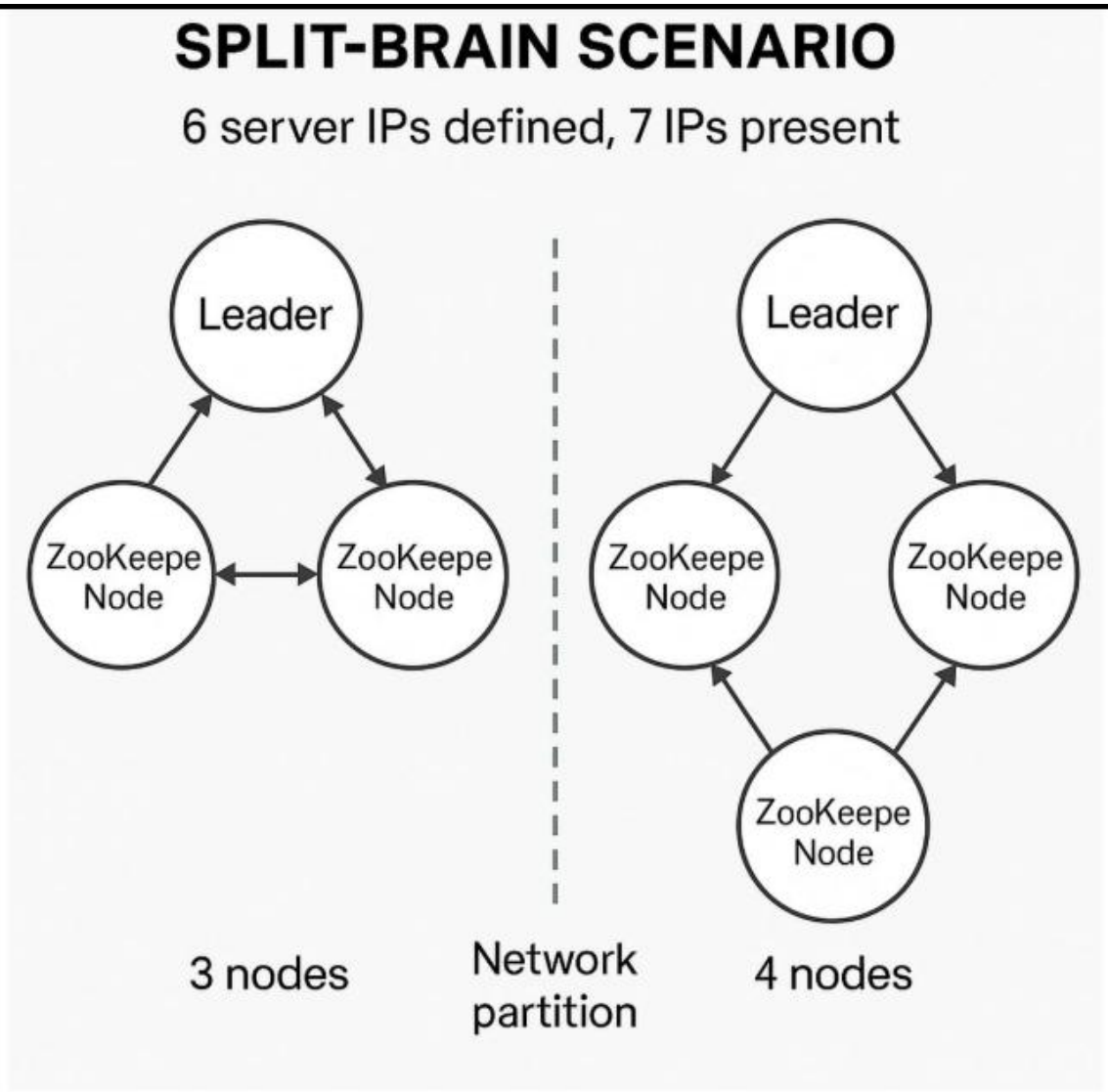


Figure 3.3: Zookeeper-Based Coordination and Leader Election Process.

The flowchart shows how Zookeeper manages coordination in the distributed system. Brokers register with Zookeeper, which monitors their status. In case of failure, Zookeeper detects the issue and initiates leader re-election. Consumer groups are dynamically updated, ensuring system stability and fault tolerance.

3.4.6 Z-Node Structure and Metadata Management

Zookeeper organizes data in a hierarchical structure called znodes. Typical structure used in the system:

- /brokers → Stores active broker information
- /topics → Maintains topic and partition metadata
- /consumers → Tracks consumer groups and assignments

Each znode stores small pieces of data that help in coordination and system state tracking.

3.4.7 Integration Workflow

The coordination module interacts with other system components as follows:

1. The messaging engine registers brokers and partitions with Zookeeper.
2. Zookeeper maintains and updates metadata continuously.
3. Consumers query Zookeeper to obtain partition assignments.
4. In case of failures, Zookeeper triggers reconfiguration.
5. Updated metadata is propagated to all relevant components.

This workflow ensures consistency and synchronization across the distributed system.

3.4.8 Advantages of Using Zookeeper

- Provides a reliable and centralized coordination mechanism
- Simplifies distributed system management
- Enables automatic failover and recovery
- Ensures data consistency across nodes
- Supports dynamic scaling of system components

Annotation

This module acts as the **coordination and control layer** of the system. It ensures synchronization, fault tolerance, and efficient management of distributed components, making the messaging system robust and scalable.

3.5 Frontend Chat Dashboard Module

The Frontend Chat Dashboard Module represents the user interaction layer of the system, providing an intuitive and responsive interface for real-time communication. This module enables users to send and receive messages seamlessly while interacting with the distributed messaging backend. It is designed to be lightweight, visually appealing, and capable of handling continuous data updates without performance degradation.

The frontend is developed using HTML, Vanilla JavaScript, and Tailwind CSS, ensuring simplicity, flexibility, and modern UI design. The application is served using the Vite development server, which offers fast rendering and efficient development workflows.

3.5.1 User Interface Design

The chat interface is designed with a modern, responsive layout inspired by contemporary messaging platforms.

- Displays chat messages in a conversational format
- Differentiates between sent and received messages
- Includes input fields for message composition
- Provides a clean and minimalistic user experience using Tailwind CSS

The design ensures usability across different screen sizes and enhances user engagement.

3.5.2 Communication with Backend

The frontend communicates with the backend through REST API calls handled by the API Gateway.

- Uses the Fetch API to send HTTP requests
- Sends messages using POST requests to the backend
- Retrieves messages using GET requests

Example interaction:

- Sending message → POST /api/messages
- Fetching messages → GET /api/messages

This ensures structured and standardized communication between the frontend and backend layers.

3.5.3 Asynchronous Polling Mechanism

To simulate real-time communication, the system uses an asynchronous polling approach.

- The frontend periodically sends requests to the backend at fixed intervals
- New messages are fetched and displayed without reloading the page
- Polling ensures that users receive updates in near real-time

This method is simple to implement and works effectively for prototype-level systems.

3.5.4 Data Rendering and UI Updates

Once messages are received from the backend:

- The frontend parses the JSON response
- Messages are dynamically added to the chat window
- The UI updates automatically to reflect new data
- Scroll management ensures that the latest messages are visible

This dynamic rendering provides a smooth and interactive user experience.

3.5.5 Performance Considerations

The frontend is optimized for performance and responsiveness:

- Lightweight framework ensures fast loading times
- Minimal dependencies reduce overhead
- Efficient DOM updates prevent UI lag
- Vite development server enables fast refresh cycles

These optimizations ensure that the interface remains responsive even during continuous updates.

3.5.6 Integration with System Modules

The frontend module integrates with other components as follows:

- API Gateway: Sends and receives HTTP requests
- Messaging Engine: Indirectly interacts through backend APIs
- Storage Module: Retrieves persisted messages via backend
- Coordination Module: Not directly visible but supports backend operations

This integration ensures a complete end-to-end communication flow.

3.5.7 Limitations and Future Enhancements

While the current implementation uses polling, future improvements may include:

- WebSocket-based real-time communication
- Push notifications for instant updates
- Advanced UI features such as typing indicators and message status

These enhancements can further improve user experience and system efficiency.

Annotation

This module acts as the presentation layer of the system. It provides a user-friendly interface for interacting with the distributed messaging backend and demonstrates the practical application of real-time communication.

3.6 Tiered Storage Module (Cloud Archival Simulation)

The Tiered Storage Module is designed to optimize storage management by separating frequently accessed data from older, less frequently used data. In large-scale messaging systems, continuous data generation leads to rapid growth of log files, which can impact system performance and storage efficiency. To address this challenge, the proposed system implements a tiered storage mechanism that archives older log segments into a simulated cloud storage environment.

This module enhances scalability and storage efficiency by maintaining active data locally for fast access while offloading older data to secondary storage.

3.6.1 Concept of Tiered Storage

Tiered storage is a data management strategy where data is stored across different storage layers based on access frequency.

- **Hot Data:** Recently generated messages stored in local log files for quick access
- **Cold Data:** Older messages archived in secondary storage
- **Separation of storage layers improves system performance and resource utilization**

This approach is widely used in distributed streaming systems to handle large-scale data efficiently.

3.6.2 Implementation Approach

The system simulates cloud storage using a dedicated directory structure within the local environment.

- Older log segments are identified based on size or age thresholds
- These segments are moved from the primary storage directory to a simulated cloud directory (".cloud_bucket/")
- The archival process is automated and runs periodically
- Metadata is updated to track the location of archived segments

Although the cloud environment is simulated, the design closely mirrors real-world cloud storage systems such as AWS S3.

3.6.3 Archival Workflow

The tiered storage module follows a structured workflow:

1. The system monitors log segment size and age
2. When a segment reaches the defined threshold, it is marked for archival
3. The segment is safely moved to the cloud storage directory
4. References to the archived segment are maintained for future access
5. Active storage is freed for new incoming data

This workflow ensures efficient data lifecycle management.

3.6.4 Data Retrieval from Archived Storage

Even after archival, the system ensures that data remains accessible:

- Consumers can request older messages
- The system checks whether the data resides in local storage or archived storage
- If archived, the system retrieves data from the cloud directory
- Data is then processed and returned to the consumer

This ensures transparency in data access, regardless of storage location.

3.6.5 Benefits of Tiered Storage

The implementation of tiered storage provides several advantages:

- Reduces storage pressure on local disks
- Improves performance by keeping active data readily accessible
- Enables efficient handling of large data volumes
- Supports long-term data retention

- Mimics real-world distributed storage strategies

3.6.6 Integration with Other Modules

The tiered storage module interacts with:

- **Log Storage Module:** Monitors and moves log segments
- **Messaging Engine:** Maintains metadata about data location
- **Consumers:** Retrieves archived data when required

This integration ensures seamless operation across storage layers.

3.6.7 Limitations

Since the cloud storage is simulated:

- It does not include real network latency or distributed storage behaviour
- Security and access control mechanisms are not implemented
- Scalability is limited to local system resources

However, the design provides a strong foundation for future integration with real cloud platforms.

Annotation

This module acts as the data lifecycle management layer of the system. It ensures efficient storage utilization by separating active and archival data, thereby enhancing system scalability and performance.

Chapter 04

TESTING AND RESULTS

4.1 Unit Testing

Unit testing is the initial phase of system validation where individual components of the system are tested independently to ensure that each module performs its intended functionality correctly. In this project, unit testing was conducted on all major components, including the messaging engine, API Gateway, log storage module, coordination module, and frontend interface. The goal of this phase was to identify and resolve issues at the component level before integrating the system.

4.1.1 Messaging Engine Testing

The custom messaging engine was tested to verify its core functionalities such as topic creation, partition assignment, message routing, and offset management.

- Messages were successfully categorized into topics and distributed across partitions.
- Sequential ordering of messages within each partition was maintained correctly.
- Offset values were incremented accurately for each incoming message.
- Concurrent message handling was tested to ensure stability under multiple inputs.

Test results confirmed that the messaging engine handled message flow efficiently without data inconsistency.

4.1.2 API Gateway Testing

The API Gateway module was tested using tools such as browser-based requests and API testing utilities.

- “POST /api/messages” endpoint successfully received and forwarded messages to the backend.
- “GET /api/messages” endpoint returned stored messages in correct JSON format.
- Invalid requests (missing fields, incorrect JSON) were properly rejected.
- HTTP status codes were returned correctly (200 OK, 400 Bad Request).

This ensured that the communication interface between frontend and backend was reliable and error resistant.

```
Apr 16, 2026 12:52:04 PM com.simplekafka.broker.SimpleKafkaBroker handleFetchRequest
INFO: Fetch request for topic: chat-topic, partition: 0, offset: 12, maxBytes: 1048576
Apr 16, 2026 12:52:04 PM com.simplekafka.broker.SimpleKafkaBroker acceptConnections
INFO: Accepted connection from /127.0.0.1:52110
Apr 16, 2026 12:52:04 PM com.simplekafka.broker.SimpleKafkaBroker handleFetchRequest
INFO: Fetch request for topic: chat-topic, partition: 1, offset: 25, maxBytes: 1048576
Apr 16, 2026 12:52:04 PM com.simplekafka.broker.SimpleKafkaBroker acceptConnections
INFO: Accepted connection from /127.0.0.1:52111
Apr 16, 2026 12:52:04 PM com.simplekafka.broker.SimpleKafkaBroker handleFetchRequest
INFO: Fetch request for topic: chat-topic, partition: 2, offset: 11, maxBytes: 1048576
Apr 16, 2026 12:52:05 PM com.simplekafka.broker.SimpleKafkaBroker acceptConnections
INFO: Accepted connection from /127.0.0.1:52112
Apr 16, 2026 12:52:05 PM com.simplekafka.broker.SimpleKafkaBroker handleFetchRequest
INFO: Fetch request for topic: chat-topic, partition: 0, offset: 12, maxBytes: 1048576
Apr 16, 2026 12:52:05 PM com.simplekafka.broker.SimpleKafkaBroker acceptConnections
INFO: Accepted connection from /127.0.0.1:52113
Apr 16, 2026 12:52:05 PM com.simplekafka.broker.SimpleKafkaBroker handleFetchRequest
INFO: Fetch request for topic: chat-topic, partition: 1, offset: 25, maxBytes: 1048576
Apr 16, 2026 12:52:05 PM com.simplekafka.broker.SimpleKafkaBroker acceptConnections
INFO: Accepted connection from /127.0.0.1:52114
Apr 16, 2026 12:52:05 PM com.simplekafka.broker.SimpleKafkaBroker handleFetchRequest
INFO: Fetch request for topic: chat-topic, partition: 2, offset: 11, maxBytes: 1048576
Apr 16, 2026 12:52:05 PM com.simplekafka.broker.SimpleKafkaBroker acceptConnections
INFO: Accepted connection from /127.0.0.1:52115
Apr 16, 2026 12:52:05 PM com.simplekafka.broker.SimpleKafkaBroker handleFetchRequest
INFO: Fetch request for topic: chat-topic, partition: 0, offset: 12, maxBytes: 1048576
Apr 16, 2026 12:52:05 PM com.simplekafka.broker.SimpleKafkaBroker acceptConnections
INFO: Accepted connection from /127.0.0.1:52116
Apr 16, 2026 12:52:05 PM com.simplekafka.broker.SimpleKafkaBroker handleFetchRequest
INFO: Fetch request for topic: chat-topic, partition: 1, offset: 25, maxBytes: 1048576
```

```
Broker 2 - java -cp target\bui
INFO: Accepted connection from /127.0.0.1:60811
12:51:04.967 [main-SendThread(localhost:2181)] DEBUG org.apache.zookeeper.ClientCnxn - Got ping response for session id:
 0x1000037f67b0001 after 2ms.
Apr 16, 2026 12:51:09 PM com.simplekafka.broker.SimpleKafkaBroker acceptConnections
INFO: Accepted connection from /127.0.0.1:51484
Apr 16, 2026 12:51:10 PM com.simplekafka.broker.SimpleKafkaBroker acceptConnections
INFO: Accepted connection from /127.0.0.1:62791
Apr 16, 2026 12:51:11 PM com.simplekafka.broker.SimpleKafkaBroker acceptConnections
INFO: Accepted connection from /127.0.0.1:58018
12:51:14.970 [main-SendThread(localhost:2181)] DEBUG org.apache.zookeeper.ClientCnxn - Got ping response for session id:
 0x1000037f67b0001 after 2ms.
Apr 16, 2026 12:51:20 PM com.simplekafka.broker.SimpleKafkaBroker lambda$start$1
INFO: Running Log Cleaner Service...
12:51:24.973 [main-SendThread(localhost:2181)] DEBUG org.apache.zookeeper.ClientCnxn - Got ping response for session id:
 0x1000037f67b0001 after 4ms.
12:51:34.976 [main-SendThread(localhost:2181)] DEBUG org.apache.zookeeper.ClientCnxn - Got ping response for session id:
 0x1000037f67b0001 after 2ms.
12:51:44.998 [main-SendThread(localhost:2181)] DEBUG org.apache.zookeeper.ClientCnxn - Got ping response for session id:
 0x1000037f67b0001 after 7ms.
12:51:54.997 [main-SendThread(localhost:2181)] DEBUG org.apache.zookeeper.ClientCnxn - Got ping response for session id:
 0x1000037f67b0001 after 2ms.
12:52:05.003 [main-SendThread(localhost:2181)] DEBUG org.apache.zookeeper.ClientCnxn - Got ping response for session id:
 0x1000037f67b0001 after 4ms.
12:52:15.014 [main-SendThread(localhost:2181)] DEBUG org.apache.zookeeper.ClientCnxn - Got ping response for session id:
 0x1000037f67b0001 after 2ms.
Apr 16, 2026 12:52:20 PM com.simplekafka.broker.SimpleKafkaBroker lambda$start$1
INFO: Running Log Cleaner Service...
12:52:25.030 [main-SendThread(localhost:2181)] DEBUG org.apache.zookeeper.ClientCnxn - Got ping response for session id:
 0x1000037f67b0001 after 3ms.
```

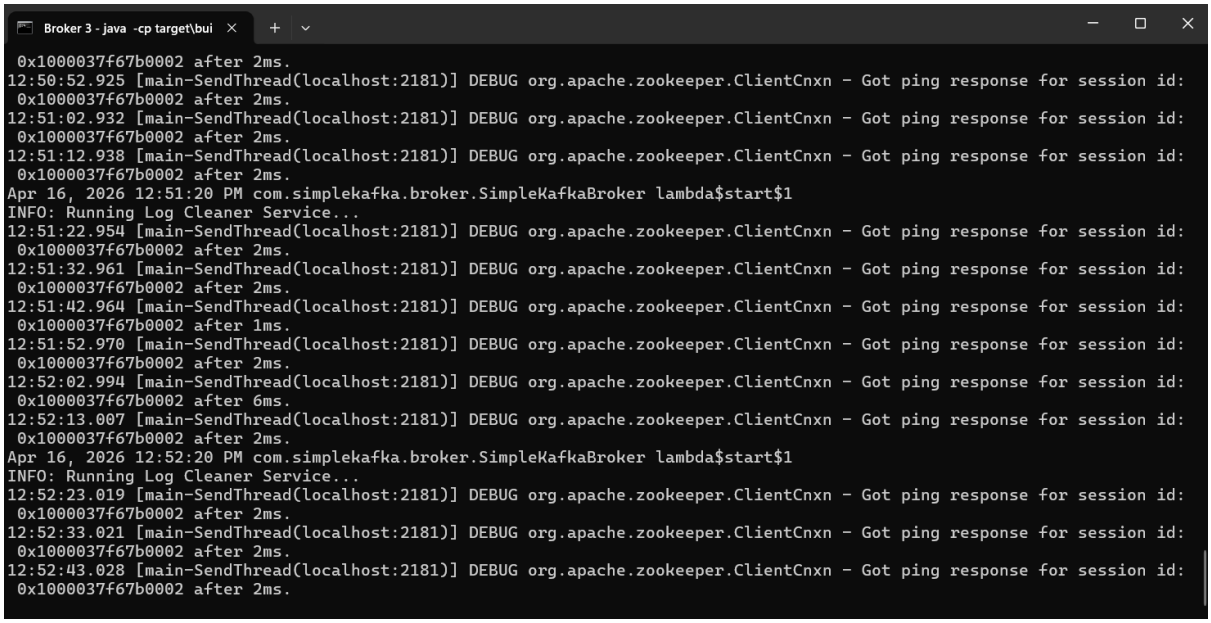


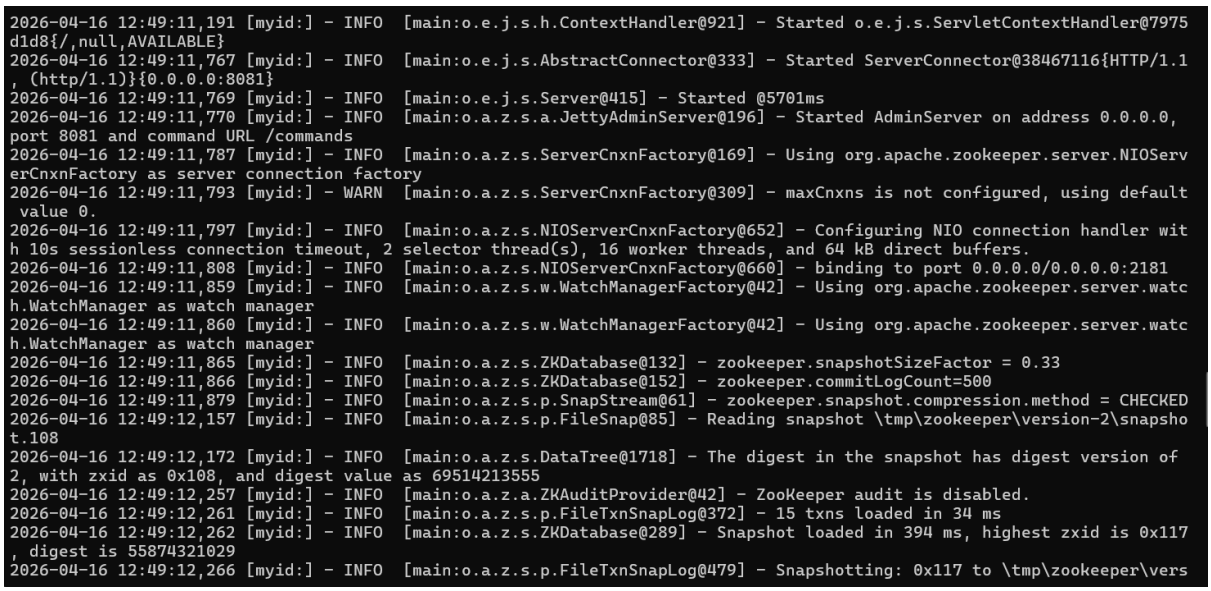
Figure 4.2: API Responses.

4.1.3 Log Storage Module Testing

The log storage system was tested to verify data persistence and retrieval mechanisms.

- Messages were correctly appended to .log files without overwriting existing data.
- .index files accurately mapped offsets to physical storage locations.
- File-Channel operations successfully handled large volumes of sequential writes.
- Data integrity was maintained even after multiple write operations.

The testing confirmed that the append-only storage mechanism was functioning efficiently and reliably.



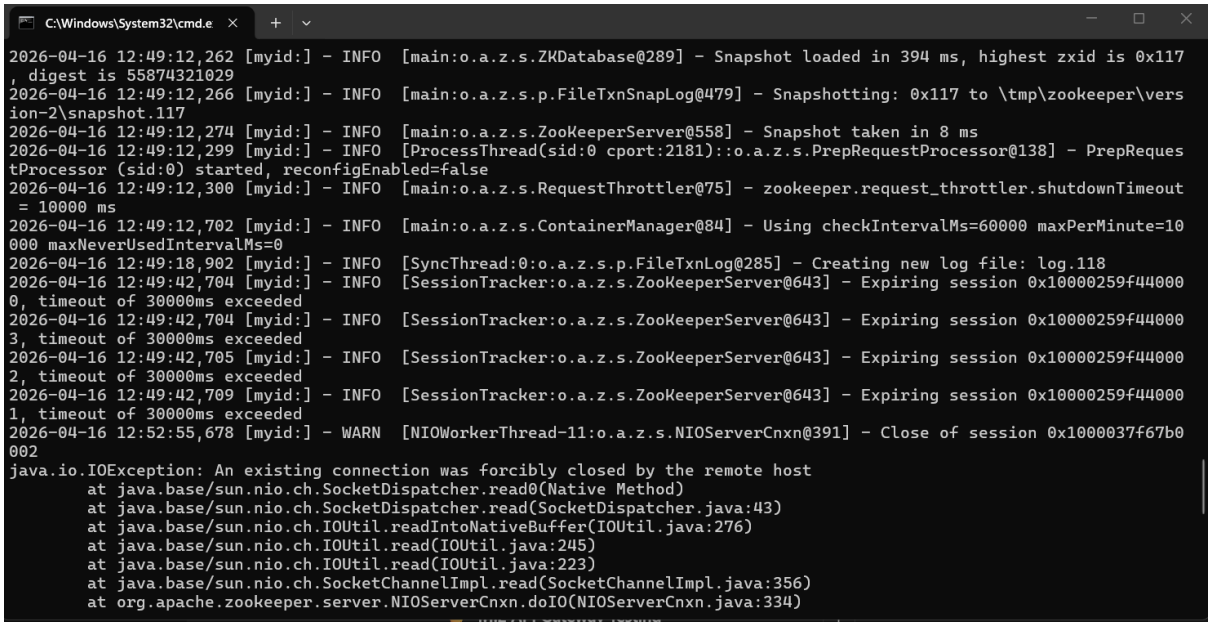


Figure 4.3 Log Storage Structure.

4.1.4 Zookeeper Coordination Testing

The distributed coordination module was tested to ensure proper handling of metadata and system synchronization.

- Broker registration and discovery were verified through Zookeeper znodes.
- Partition leadership assignment was tested under normal conditions.
- Simulated node failure scenarios triggered leader re-election successfully.
- Consumer group coordination ensured balanced partition distribution.

These tests validated the system's ability to maintain consistency and fault tolerance in a distributed environment.

4.1.5 Frontend Module Testing

The frontend chat interface was tested for usability and real-time interaction.

- Messages entered by users were successfully sent to the backend.
- Polling mechanism fetched new messages at regular intervals.
- UI updated dynamically without requiring page refresh.
- Message rendering was accurate and consistent across sessions.

The frontend demonstrated smooth interaction with the backend system.

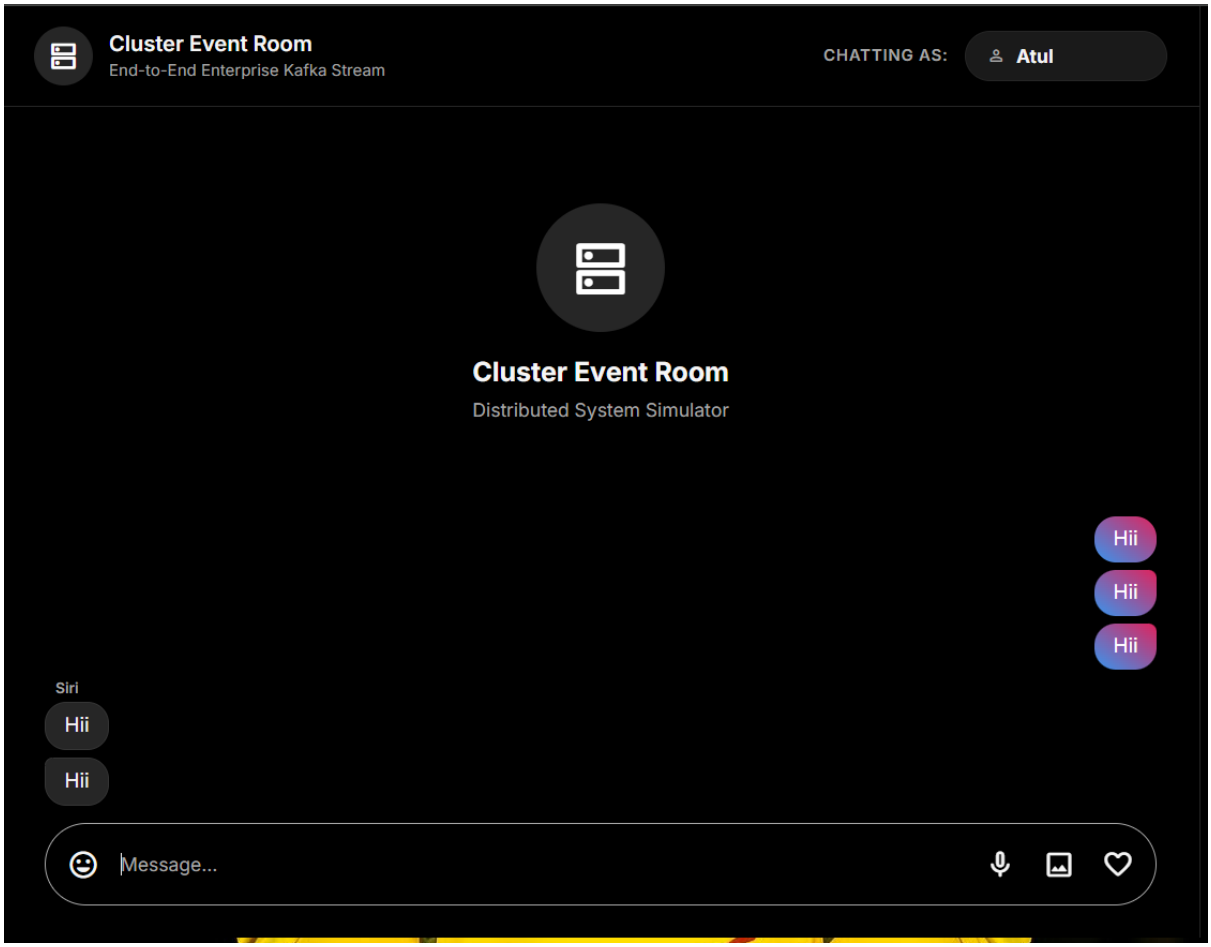


Figure 4.1: Frontend Chat Interface.

4.1.6 Tiered Storage Module Testing

The tiered storage mechanism was tested to validate archival functionality.

- Log segments exceeding defined thresholds were successfully moved to the ".cloud_bucket/" directory.
- Archived data remained accessible when requested by consumers.
- Metadata tracking ensured correct identification of storage locations.

This confirmed that the system could efficiently manage data lifecycle without affecting active operations.

Annotation

Unit testing ensured that each individual module of the system operated correctly in isolation. By validating components such as the messaging engine, API Gateway, storage layer, coordination module, frontend, and archival system independently, the project established a strong foundation for subsequent integration and system-level testing.

4.2 Integration Testing

Integration testing focuses on verifying the interaction and data flow between different modules of the system. After ensuring that individual components function correctly through unit testing, this phase validates that all modules work together seamlessly as a complete distributed messaging system. The primary objective is to ensure that data is correctly transmitted across components without loss, delay, or inconsistency.

In this project, integration testing was performed by combining the API Gateway, messaging engine, log storage module, Zookeeper coordination, and frontend interface to simulate real-world usage scenarios.

4.2.1 End-to-End Message Flow Testing

The complete communication pipeline was tested to ensure proper data flow across all layers.

Workflow Tested:

1. User sends a message through the frontend interface
2. API Gateway receives the request
3. Message is forwarded to the messaging engine
4. Messaging engine assigns topic and partition
5. Message is stored in append-only log using File-Channel
6. Consumer retrieves message based on offset
7. Frontend fetches message via API and displays it

- All steps executed successfully without data loss
- Messages were delivered in correct order
- No inconsistencies observed in message flow

This confirmed that the system supports seamless end-to-end communication.

4.2.2 API Gateway and Messaging Engine Integration

The interaction between the API Gateway and messaging engine was tested.

- API requests were correctly forwarded to the messaging engine
- Message payloads were processed without errors
- Response handling was consistent and reliable
- System handled multiple consecutive requests efficiently

This ensured smooth communication between the interface layer and core processing module.

4.2.3 Messaging Engine and Storage Integration

This test validated the interaction between the messaging engine and log storage system.

- Messages were successfully written to correct partition log files
- Offset values were updated accurately after each write
- No data corruption occurred during storage operations
- Retrieval from storage matched the stored data exactly

This confirmed the reliability of the persistence layer.

4.2.4 Zookeeper and Messaging Engine Integration

The coordination between Zookeeper and the messaging engine was tested.

- Brokers were successfully registered in Zookeeper
- Partition metadata was correctly maintained
- Leader election worked during simulated failures
- Consumer group assignments were updated dynamically

This ensured proper synchronization and coordination in the distributed environment.

4.2.5 Frontend and Backend Integration

The interaction between the frontend interface and backend services was tested.

- Messages sent from the frontend reached the backend successfully
- Polling mechanism retrieved new messages correctly
- UI updated dynamically with new data
- No delays or failures observed during continuous interaction

This validated the complete user interaction cycle.

4.2.6 Concurrent Operation Testing

The system was tested under multiple simultaneous interactions.

- Multiple users sending messages concurrently
- Continuous read and write operations
- No message duplication or loss occurred
- System maintained stability under load

This demonstrated the system's ability to handle real-world usage scenarios.

Annotation

Integration testing confirmed that all modules of the system work cohesively as a unified architecture. The successful validation of end-to-end workflows, inter-module communication, and concurrent operations demonstrates that the system can handle real-time messaging efficiently in a distributed environment.

4.3 System Testing

System testing is performed to evaluate the complete and integrated system under realistic operating conditions. Unlike unit and integration testing, which focus on individual components and their interactions, system testing validates the overall functionality, performance, and stability of the entire application as a single unit.

In this project, system testing was conducted to ensure that the distributed messaging system operates reliably under continuous usage, handles real-time communication efficiently, and meets the defined functional and non-functional requirements.

4.3.1 Real-Time Messaging Test

The system was tested for real-time communication capabilities by simulating continuous user interaction.

- Messages were sent from the frontend at regular intervals
- The backend processed and stored messages without delay
- The frontend polling mechanism successfully fetched new messages
- The chat interface updated dynamically in near real-time

The system demonstrated smooth message flow with minimal latency, confirming its suitability for real-time applications.

4.3.2 Continuous Operation Test

The system was executed continuously for an extended period to evaluate stability.

- The messaging engine handled continuous message ingestion without crashes
- API Gateway remained responsive throughout execution
- Log storage maintained consistent write operations
- No memory leaks or performance degradation were observed

This confirmed that the system is stable under prolonged usage.

4.3.3 Load Handling Test

The system was tested under multiple message inputs to evaluate its performance under load.

- Multiple messages were sent in rapid succession
- The system handled concurrent read/write operations efficiently
- Partition-based processing ensured balanced load distribution
- No message loss or duplication occurred

This demonstrated the system's ability to handle increased workload effectively.

4.3.4 Fault Tolerance Test

Fault scenarios were simulated to evaluate system resilience.

- Broker/node failure was simulated

- Zookeeper successfully triggered leader re-election
- System resumed operation without data loss
- Consumers continued reading messages after recovery

This validated the fault tolerance capability of the distributed system.

4.3.5 Data Consistency Test

The system was tested to ensure consistency of stored and retrieved data.

- Messages retrieved from storage matched the original input
- Order of messages within partitions was preserved
- Offset tracking ensured correct sequence of consumption
- No data corruption was observed

This confirmed that the system maintains strong data consistency.

4.3.6 End-User Experience Test

The system was evaluated from a user perspective.

- Chat interface remained responsive during continuous usage
- Messages appeared in correct order and format
- No UI glitches or delays were observed
- Overall user interaction was smooth and intuitive

This ensured that the system provides a satisfactory user experience.

Annotation

System testing validated the performance, reliability, and robustness of the complete distributed messaging system. The successful execution of real-time communication, continuous operation, fault tolerance, and load handling tests confirms that the system can operate effectively in real-world scenarios.

4.4 Validation

Validation is the process of ensuring that the developed system meets the requirements and objectives defined during the analysis phase. While testing focuses on identifying errors and verifying functionality, validation confirms that the system fulfils its intended purpose and behaves as expected in real-world scenarios.

In this project, validation was carried out by comparing the system's actual performance and behaviour with the predefined functional and non-functional requirements. The objective was to ensure that the distributed messaging system operates correctly, efficiently, and reliably under practical conditions.

4.4.1 Requirement Validation

The system was evaluated against all specified requirements to ensure completeness.

- The system successfully supports real-time message transmission between users
- Topic-based messaging and partitioning are correctly implemented
- Append-only log storage ensures reliable data persistence
- REST APIs provide proper communication between frontend and backend
- Distributed coordination using Zookeeper functions as expected

All major functional requirements were successfully achieved.

4.4.2 Performance Validation

The system was validated for performance characteristics such as latency and throughput.

- Message delivery occurred with minimal delay
- API response times remained within acceptable limits
- Log storage operations were executed efficiently using sequential writes
- System maintained consistent performance under continuous usage

This confirmed that the system meets the expected performance standards for real-time communication.

4.4.3 Data Integrity Validation

Data integrity was verified to ensure accuracy and consistency of stored information.

- Messages stored in log files matched the original input data
- Offset tracking ensured correct sequence of message retrieval
- No data loss or corruption was observed during testing
- Archived data remained consistent and retrievable

This ensured that the system maintains reliable data handling.

4.4.4 Fault Tolerance Validation

The system was validated for its ability to handle failures gracefully.

- Leader re-election occurred successfully during simulated failures
- System continued operation without manual intervention
- Data remained accessible after recovery
- Consumer processes resumed correctly

This demonstrated the robustness of the distributed architecture.

4.4.5 User Interaction Validation

The system was tested from an end-user perspective to validate usability.

- Users were able to send and receive messages without errors
- The frontend interface updated dynamically with new messages
- No significant delays or inconsistencies were observed

- Overall interaction remained smooth and intuitive

This confirmed that the system delivers a functional and user-friendly experience.

Annotation

Validation ensured that the system not only functions correctly but also meet the intended design goals and performance expectations. The successful validation of requirements, performance, data integrity, and user interaction confirms that the proposed distributed messaging system is both effective and reliable.

4.5 Results

The results of the implemented system demonstrate the effectiveness of the proposed distributed messaging architecture in handling real-time chat communication. Through comprehensive testing and validation, the system exhibited strong performance in terms of message delivery, storage efficiency, fault tolerance, and overall system stability.

4.5.1 Performance Outcomes

The system achieved efficient performance across multiple operational parameters:

- Message transmission from frontend to backend occurred with minimal latency
- API response time remained consistently low during continuous operation
- Log storage operations were executed efficiently due to sequential disk writes
- System handled concurrent message requests without degradation in performance

These results indicate that the system can support real-time communication with high responsiveness.

4.5.2 Message Delivery and Consistency

The system ensured reliable message delivery and maintained data consistency:

- Messages were delivered in correct order within each partition
- No message loss or duplication was observed during testing
- Offset-based tracking ensured accurate message retrieval
- Stored data matched exactly with the original input

This confirms that the system maintains strong consistency and reliability.

4.5.3 Storage Efficiency

The log-based storage mechanism proved to be highly efficient:

- Append-only log structure enabled fast write operations
- Disk I/O overhead was minimized due to sequential access patterns
- Tiered storage reduced load on primary storage by archiving older data
- System handled continuous data growth without performance issues

This demonstrates the effectiveness of replacing traditional databases with log-based storage.

4.5.4 Fault Tolerance and Recovery

The system successfully handled failure scenarios:

- Leader re-election ensured uninterrupted system operation
- Data remained intact during simulated node failures
- Consumers resumed message consumption without data inconsistency
- System recovered automatically without manual intervention

These results validate the robustness of the distributed architecture.

4.5.5 User Interface Results

The frontend interface provided a smooth and responsive user experience during system testing.

- Messages were displayed in real time without requiring page refresh
- The chat interface maintained correct message order and formatting
- UI updates were handled dynamically through asynchronous polling
- No delays or rendering issues were observed during continuous interaction

These results confirm that the system delivers an efficient and user-friendly communication experience.

4.5.6 Overall System Evaluation

Based on testing and validation, the system successfully demonstrates:

- A fully functional distributed messaging backend
- Efficient real-time communication capability
- High scalability through partitioned architecture
- Reliable data persistence using append-only logs
- Effective coordination using Zookeeper

The results confirm that the system meets all defined objectives and performs reliably under different operating conditions.

Annotation

The results validate that the proposed system can handle real-time messaging efficiently while maintaining performance, reliability, and scalability. The successful implementation of a custom Kafka-like messaging engine demonstrates the feasibility of building distributed event streaming systems from scratch.

Chapter 05

CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

The development of a scalable and efficient real-time messaging system is a critical requirement in modern distributed applications. This project successfully demonstrates the design and implementation of a custom Kafka-like distributed messaging engine for real-time chat applications, built entirely from scratch using Java. The system integrates multiple components, including a REST-based API Gateway, a partitioned messaging engine, an append-only log storage mechanism, distributed coordination using Zookeeper, and a responsive frontend interface.

The project achieved all its primary objectives by implementing a fully functional event-driven architecture that supports asynchronous communication between producers and consumers. The use of append-only log storage using Java NIO File-Channel proved to be highly effective in ensuring high-throughput data ingestion and reliable message persistence. By eliminating dependency on traditional databases, the system demonstrates a more efficient approach to handling streaming data workloads.

The integration of Apache Zookeeper enabled robust distributed coordination, including broker management, partition leadership, and consumer group assignment. This significantly improved the system's fault tolerance and ensured continuity of operations even in failure scenarios. Additionally, the implementation of partitioning and offset-based message retrieval allowed for scalable and ordered data processing.

The frontend chat interface successfully demonstrated real-time user interaction through asynchronous polling, providing a smooth and responsive user experience. The inclusion of a tiered storage mechanism further enhanced the system by enabling efficient data lifecycle management through archival of older log segments.

Overall, the system performed reliably during testing, maintaining low latency, high throughput, and data consistency under continuous operation. The project not only validates the feasibility of building a distributed messaging system from scratch but also provides a deep understanding of core concepts such as event streaming, partitioning, replication, and distributed coordination.

This work serves as a strong foundation for developing large-scale, production-ready messaging systems and highlights the importance of distributed system design in modern software engineering.

5.2 Future Scope

Although the proposed system successfully demonstrates a scalable and efficient distributed messaging architecture for real-time chat applications, there are several areas where further enhancements can be made to extend its capabilities and bring it closer to a production-grade system.

One of the primary improvements can be the implementation of **WebSocket-based communication** to replace the current polling mechanism. This would enable true real-time bidirectional communication between the frontend and backend, significantly reducing latency and improving system efficiency.

The system can also be extended to support **full-scale cloud deployment** using platforms such as AWS, Google Cloud, or Azure. Integrating real cloud storage services like Amazon S3 instead of the simulated. “cloud_bucket/” directory would enhance scalability, reliability, and accessibility across distributed environments.

Another important enhancement is the incorporation of **advanced security mechanisms**. Features such as JWT-based authentication, role-based access control, and encrypted communication (HTTPS/TLS) can be implemented to ensure secure data transmission and user authentication.

The messaging engine can be further improved by implementing **advanced replication strategies**, such as configurable replication factors, in-sync replica (ISR) management, and stronger consistency guarantees. This would increase fault tolerance and make the system more robust in large-scale distributed deployments.

Additionally, the system can be enhanced with **real-time analytics and monitoring tools**, allowing administrators to track message throughput, system health, and performance metrics. Integration with monitoring platforms can provide insights into system behavior and help in performance optimization.

From a user experience perspective, the frontend can be upgraded to include features such as **typing indicators, message delivery status (sent/seen), multimedia support, and user profiles**, making it comparable to modern chat applications.

Another promising direction is the integration of **machine learning models** for intelligent message processing, such as spam detection, sentiment analysis, or anomaly detection in communication patterns. This would add an intelligent layer to the system and expand its application scope.

Finally, the system can be evolved into a **fully distributed microservices architecture**, where each component (API Gateway, messaging engine, storage, coordination) operates as an independent service deployed using containerization technologies such as Docker and orchestration tools like Kubernetes.

In summary, while the current system provides a strong and functional prototype of a distributed messaging platform, these future enhancements can significantly improve its scalability, security, usability, and real-world applicability.

REFERENCES

1. **Apache Kafka Documentation**
   Available: https://kafka.apache.org/documentation/
2. **Apache Zookeeper Documentation**
   Available: https://zookeeper.apache.org/doc/
3. **Oracle Java Documentation (JDK 11)**
   Available: https://docs.oracle.com/en/java/javase/11/
4. **Java NIO File-Channel Documentation**
   Available:
   https://docs.oracle.com/javase/8/docs/api/java/nio/channels/FileChannel.html
5. **Javalin Framework Documentation**
   Available: https://javalin.io/documentation
6. **Tailwind CSS Documentation**
   Available: https://tailwindcss.com/docs
7. **Vite Development Server Documentation**
   Available: https://vitejs.dev/guide/
8. **MDN Web Docs – Fetch API**
   Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
9. **Designing Data-Intensive Applications – Martin Kleppmann**
   O'Reilly Media, 2017
10. **Distributed Systems Concepts and Design – George Coulouris et al.**
    Pearson Education
11. **Research Paper: Kafka: a Distributed Messaging System for Log Processing**
    Available: https://www.microsoft.com/en-us/research/publication/kafka-a-distributed-messaging-system-for-log-processing/
12. **Confluent Blog – Kafka Architecture Explained**
    Available: https://www.confluent.io/blog/